Tool Survey Proposal — T07: CI/CD & Test-Harness Engineering

I. Nhóm sinh viên thực hiện

MSSV	Họ và tên	Đóng góp
23120242	Nguyễn Văn Bình Dương	100%
23120409	Nguyễn Khắc Vượng	100%
23120410	Trần Quốc Vỹ	100%

Project dự định demo: EShop.

Mục tiêu: chạy unit/API/smoke test trên PR, lưu artefact/coverage và so sánh người với AI khi phân tích flaky test.

II. Nội dung khảo sát các công cụ

Trong phát triển phần mềm, mã nguồn thường xuyên được cập nhật và tái cấu trúc bởi nhiều thành viên. Mỗi thay đổi có thể tạo ra lỗi mới hoặc ảnh hưởng đến các chức năng đã hoạt động ổn định. Nếu chỉ kiểm thử thủ công trước khi bàn giao, nhóm phát triển có thể phát hiện lỗi muộn và mất nhiều thời gian để xác định nguyên nhân gốc rễ.

Vì vậy, việc tự động hóa quá trình xây dựng và kiểm thử là cần thiết. Nhóm khảo sát GitHub Actions làm công cụ CI chính, Jenkins làm phương án dự phòng khi không thể dùng GitHub Actions, và ChatGPT để phân tích log cùng retry history.

A. GitHub Actions

1. Tổng quan

GitHub Actions là nền tảng tự động hóa tích hợp trong GitHub, hỗ trợ xây dựng các quy trình CI/CD để build, test và triển khai phần mềm. Ngoài ra, công cụ còn có thể tự động hóa hoạt động quản lý repository, chẳng hạn xử lý issue hoặc gán nhãn.

2. Các thành phần chính

Thành phần	Ý nghĩa
Workflow	Quy trình tự động hóa, được khai báo trong file YAML tại thư mục `.github/workflows`.
Event	Sự kiện kích hoạt workflow, ví dụ `push` hoặc `pull_request`.
Job	Nhóm công việc gồm nhiều bước, được thực hiện trên một runner.
Step	Một bước thực thi, có thể chạy lệnh hoặc gọi action.
Action	Thành phần tái sử dụng để thực hiện nhiệm vụ cụ thể.

Thành phần	Ý nghĩa
Runner	Máy thực thi các job, do GitHub cung cấp hoặc người dùng tự quản lý.

Một workflow có thể gồm nhiều job. Các job độc lập có thể chạy song song; nếu cần thứ tự thực hiện, người dùng khai báo quan hệ phụ thuộc giữa các job.

Ví dụ: workflow kiểm thử có thể gồm một job với các step: lấy mã nguồn, thiết lập môi trường, cài thư viện và chạy test.

3. Cơ chế hoạt động

GitHub Actions hoạt động dựa trên cấu hình workflow và các sự kiện được khai báo. File workflow sử dụng đuôi .yml hoặc .yaml.

Trong tình huống kiểm thử một thay đổi mã nguồn, quy trình được thiết kế như sau:

1. Lập trình viên tạo pull request.
2. Workflow kiểm thử được kích hoạt.
3. Runner chuẩn bị môi trường và lấy mã nguồn.
4. Các lệnh kiểm thử được thực hiện.
5. Nhóm phát triển xem kết quả, sửa lỗi và chạy lại nếu cần.

Ví dụ, một thay đổi trong chức năng tính tổng tiền giỏ hàng cần được kiểm tra với các trường hợp: giỏ hàng rỗng, nhiều sản phẩm, số lượng không hợp lệ và mã giảm giá hết hạn. Kết quả kiểm thử giúp người đánh giá pull request có thêm căn cứ trước khi chấp nhận thay đổi.

4. Ví dụ workflow kiểm thử

Giả sử dự án sử dụng Node.js, có file `package-lock.json` và đã khai báo lệnh test trong `package.json`. Có thể tạo file `.github/workflows/ci.yml` với nội dung:

```yaml
name: Automated Testing

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10

    defaults:
      run:
        working-directory: backend

    steps:
      - name: Checkout source code
        uses: actions/checkout@v6

      - name: Set up Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "22"

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test
```

Ví dụ này được xây dựng theo hướng dẫn kiểm thử Node.js của GitHub, sử dụng các action để lấy mã nguồn và thiết lập môi trường trước khi chạy lệnh của dự án. Các phần chính gồm:

- on: chạy khi push lên main hoặc có pull request hướng vào main.
- permissions: giới hạn token ở quyền đọc nội dung repository.
- runs-on: chọn môi trường Ubuntu.
- timeout-minutes: giới hạn thời gian thực hiện job.
- uses: gọi action có sẵn.
- run: thực thi lệnh. Để sử dụng với EShop, các lệnh npm chạy trong thư mục `backend`; dự án còn cần bộ test thực tế. Hiện `npm test` chỉ là placeholder và chưa kiểm thử nghiệp vụ.

5. Ưu điểm và hạn chế

5.1. Ưu điểm

- **Giảm thao tác lặp lại:** nhóm không phải chạy lại toàn bộ lệnh kiểm tra bằng tay.
- **Phản hồi sớm:** lỗi được phát hiện ngay trong quá trình đề xuất thay đổi.
- **Thống nhất quy trình:** các thay đổi được kiểm tra theo cùng cấu hình.
- **Dễ chia sẻ cấu hình:** workflow được quản lý cùng mã nguồn.

5.2. Nhược điểm

- Cấu hình sai có thể khiến workflow bỏ qua kiểm tra cần thiết.
- Bộ test thiếu hoặc yếu làm giảm giá trị của kết quả thành công.
- Test không ổn định có thể gây kết quả thất bại khó tái hiện.
- Nhiều job hoặc test chậm làm tăng thời gian chờ và tài nguyên sử dụng.

6. Chi phí và bảo mật

Theo tài liệu về GitHub Actions, repository công khai được sử dụng miễn phí standard GitHub-hosted runners. Repository riêng tư có hạn mức theo gói; phần sử dụng vượt hạn mức có thể phát sinh phí. Larger runners được tính phí kể cả với repository công khai.

Về bảo mật, cần áp dụng quyền tối thiểu cho GITHUB_TOKEN, tránh ghi thông tin nhạy cảm trực tiếp trong workflow hoặc log, và kiểm tra các action được sử dụng. Khi cần cố định phiên bản action chặt chẽ, nên tham chiếu bằng commit SHA đầy đủ.

B. ChatGPT

1. Tổng quan

ChatGPT là công cụ AI nhận yêu cầu bằng ngôn ngữ tự nhiên và phân tích nội dung người dùng cung cấp. Trong Topic 7, nhóm dùng ChatGPT để **hỗ trợ triage flaky test**: đọc log từ pipeline, lịch sử chạy lại và mô tả môi trường, sau đó đề xuất giả thuyết về nguyên nhân. ChatGPT không phải CI server và không tự phát hiện flake nếu nhóm chưa cung cấp dữ liệu chạy test.

2. Dữ liệu đầu vào và kết quả mong đợi

Thành phần	Vai trò trong thí nghiệm
Log test/CI	Cho biết test nào thất bại, thông báo lỗi, thời điểm và bước pipeline liên quan.
Retry history	Cho thấy cùng một test có lúc pass, lúc fail trong các lần chạy.
Prompt	Quy định câu hỏi, phạm vi suy luận và định dạng câu trả lời của ChatGPT.
Phản hồi AI	Giả thuyết, dòng log hỗ trợ và ba bước kiểm tra tiếp theo; chưa phải kết luận cuối cùng.

3. Cơ chế áp dụng với EShop

Nhóm tạo một sleep-based flaky test có nguyên nhân đã biết và chạy 10 lần trên pipeline. Một thành viên phân tích log thủ công trước. Sau đó nhóm đưa **đúng cùng log và retry history** cho ChatGPT, không bổ sung đáp án hoặc thông tin mà người phân tích chưa có. Cuối cùng, nhóm thực hiện các bước kiểm tra được đề xuất và so sánh hai cách triage theo: nguyên nhân dự đoán có đúng không, bằng chứng có cụ thể không, bước xác minh có khả thi không và có suy đoán thiếu căn cứ không.

4. Ví dụ prompt dùng trong demo

```
Bạn đang hỗ trợ phân tích một flaky test trong CI của EShop.
Dựa CHỈ trên log và kết quả 10 lần chạy bên dưới:
1. Nêu tối đa 3 giả thuyết nguyên nhân, xếp theo mức độ hợp lý.
2. Với mỗi giả thuyết, chỉ rõ dòng log hoặc quan sát hỗ trợ.
3. Đề xuất đúng 3 bước kiểm tra tiếp theo để phân biệt các giả thuyết.
4. Nêu thông tin còn thiếu; không khẳng định root cause khi chưa đủ bằng chứng.
[Dán log đã lược bỏ token, mật khẩu và dữ liệu nhạy cảm]
[Dán bảng 10 lần chạy và retry history]
```

5. Ưu điểm và hạn chế

- **Ưu điểm:** tóm tắt log dài, gợi ý nhiều hướng điều tra, giúp nhóm tạo danh sách kiểm tra có cấu trúc.
- **Hạn chế:** có thể suy đoán từ log thiếu ngữ cảnh, nhầm flaky test với lỗi sản phẩm, hoặc đưa bước kiểm tra chung chung. Chất lượng kết quả phụ thuộc dữ liệu và prompt; kết luận phải được xác minh bằng chạy lại test.

6. Chi phí và bảo mật

ChatGPT có gói Free nhưng chịu giới hạn sử dụng; nhóm cần kiểm tra khả năng truy cập trước buổi demo. Trước khi đưa log vào công cụ, nhóm sẽ ẩn token, mật khẩu và thông tin nhạy cảm. Nhóm lưu prompt và phản hồi để đối chiếu với báo cáo phân tích thủ công.

C. Jenkins

1. Tổng quan

Jenkins là một máy chủ tự động hóa mã nguồn mở, được sử dụng để tự động hóa các công việc liên quan đến xây dựng, kiểm thử và phân phối hoặc triển khai phần mềm. Trong quy trình phát triển, Jenkins có thể đảm nhiệm việc lấy mã nguồn, gọi các lệnh build và test, sau đó tổng hợp kết quả.

Ví dụ, khi lập trình viên cập nhật chức năng giỏ hàng, một quy trình CI có thể chạy lại các test liên quan để kiểm tra thay đổi trước khi tích hợp vào nhánh chính.

2. Các thành phần chính

Jenkins sử dụng mô hình controller–agent: controller quản lý và điều phối công việc; agent thực hiện các tác vụ được giao. Việc sử dụng agent giúp phân tách hoạt động quản lý với công việc build và test.

Thành phần	Vai trò
Controller	Quản lý cấu hình, hàng đợi và điều phối công việc.
Agent	Máy hoặc môi trường thực hiện các tác vụ.
Job	Một công việc được cấu hình trong Jenkins.
Pipeline	Quy trình tự động hóa gồm nhiều công đoạn.
Stage	Một giai đoạn, chẳng hạn Checkout, Build hoặc Test.

Thành phần	Vai trò
Step	Thao tác cụ thể trong một giai đoạn.
Jenkinsfile	File chứa định nghĩa pipeline, thường được lưu cùng mã nguồn.
Plugin	Thành phần mở rộng khả năng của Jenkins.

Các khái niệm pipeline, stage và step giúp chia quy trình thành những phần rõ ràng, dễ theo dõi. Plugin bổ sung khả năng tích hợp với công cụ quản lý mã nguồn, công cụ build, dịch vụ đám mây hoặc công cụ phân tích. Người quản trị cần lựa chọn và duy trì các plugin phù hợp với nhu cầu.

3. Cơ chế hoạt động

Một quy trình kiểm thử tự động bằng Jenkins có thể được thiết kế như sau:

1. Lập trình viên cập nhật mã nguồn lên repository.
2. Jenkins nhận tín hiệu kích hoạt.
3. Controller đưa công việc vào hàng đợi và phân công agent.
4. Agent lấy mã nguồn, chuẩn bị môi trường và chạy kiểm thử.
5. Jenkins ghi nhận kết quả để nhóm phát triển kiểm tra.

Khi tích hợp GitHub, có thể cấu hình webhook và cơ chế kích hoạt phù hợp để Jenkins kiểm tra thay đổi sau sự kiện push. Việc tự động chạy cần được cấu hình; chỉ thêm Jenkinsfile vào repository chưa đủ.

Ví dụ, nhóm thay đổi công thức tính tổng tiền đơn hàng. Pipeline sẽ chạy bộ test đã chuẩn bị. Nếu công thức sai và test kiểm tra đúng trường hợp đó, kết quả sẽ chỉ ra lỗi để lập trình viên sửa lại.

4. Jenkins Pipeline và Jenkinsfile

Jenkins Pipeline cho phép mô tả quy trình tự động hóa bằng mã. Jenkins hỗ trợ hai dạng cú pháp:

- Declarative Pipeline: cấu trúc rõ ràng, thuận tiện cho người mới.
- Scripted Pipeline: linh hoạt hơn khi cần viết logic xử lý phức tạp. Định nghĩa pipeline có thể được lưu trong file Jenkinsfile tại thư mục gốc của dự án. Quản lý file này cùng mã nguồn giúp nhóm xem xét thay đổi, theo dõi lịch sử và thống nhất cấu hình quy trình. Đối với bài thực hành, Declarative Pipeline phù hợp vì dễ trình bày từng giai đoạn và giải thích nhiệm vụ của chúng.

5. Ví dụ pipeline kiểm thử

```
pipeline {
    agent { label 'linux-node' }

    options {
        skipDefaultCheckout(true)
        timeout(time: 10, unit: 'MINUTES')
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }

        stage('Install Dependencies') {
            steps { dir('backend') { sh 'npm ci' } }
        }

        stage('Test') {
            steps { dir('backend') { sh 'npm test' } }
        }
    }

    post {
        success { echo 'Kiểm thử thành công.' }
        failure { echo 'Pipeline thất bại. Kiểm tra log.' }
    }
}
```

Pipeline gồm ba giai đoạn: lấy mã nguồn → cài thư viện → chạy kiểm thử. Ví dụ giả định agent nhãn `linux-node` đã được cấu hình với Node.js/npm; lệnh chạy trong `backend` của EShop.

Cấu hình	Ý nghĩa
`agent`	Chọn môi trường chạy.
`options`	Tắt checkout mặc định và giới hạn thời gian chạy.
`stages`, `steps`	Khai báo các giai đoạn và thao tác.
`post`	Thông báo theo kết quả pipeline.

Nếu npm test trả về mã lỗi, pipeline báo thất bại. Có thể bổ sung bước junit để đọc báo cáo JUnit XML nếu bộ test có xuất file này.

6. Ưu điểm và hạn chế

6.1. Ưu điểm

Từ cơ chế hoạt động trên, có thể đánh giá Jenkins có những ưu điểm:

- Tùy chỉnh quy trình: chia công việc thành các giai đoạn theo nhu cầu.
- Chủ động môi trường: lựa chọn agent phù hợp với dự án.
- Dễ quản lý cấu hình bằng mã: Jenkinsfile có thể được xem xét cùng thay đổi phần mềm.
- Khả năng mở rộng: bổ sung tích hợp thông qua plugin.

6.2. Hạn chế

Việc sử dụng Jenkins cũng đặt ra những yêu cầu vận hành:

- Cần cài đặt và cấu hình controller, agent và kết nối repository.
- Cần duy trì môi trường chạy, phiên bản phần mềm và plugin.
- Pipeline phức tạp làm tăng công sức bảo trì.
- Test thiếu hoặc không ổn định làm giảm độ tin cậy của kết quả.
- Công việc có thể phải chờ khi agent không đủ tài nguyên.

7. Chi phí và bảo mật

Jenkins là phần mềm mã nguồn mở. Tuy nhiên, triển khai thực tế vẫn cần tính đến chi phí máy chủ, lưu trữ, mạng và nhân lực quản trị. Vì vậy, nên phân biệt chi phí phần mềm với tổng chi phí vận hành.

Jenkins cần được cấu hình xác thực và phân quyền phù hợp để kiểm soát người được phép quản lý hệ thống hoặc thực hiện công việc. Token, mật khẩu và khóa truy cập nên được quản lý bằng Jenkins Credentials, thay vì ghi trực tiếp trong Jenkinsfile. Cơ chế che thông tin trong log chỉ giảm nguy cơ lộ vô tình; không nên cấp credentials đáng tin cậy cho pipeline không đáng tin cậy.

D. So sánh 3 công cụ

Ứng viên	Chi phí giấy phép	Độ khó học	Phù hợp EShop	Khả năng AI	Cộng đồng
GitHub Actions — CI chính	Miễn phí standard runner cho repo công khai; lưu trữ có hạn	Cao (YAML)	Cao: repo ở GitHub, chạy npm	Cấp log, không tự triage	Docs/action mẫu rộng
ChatGPT — AI triage	Gói Free có giới hạn	Thấp; cần kiểm chứng	Đọc log và retry history	Giả thuyết + 3 bước kiểm tra	Docs/cộng đồng rộng
Jenkins — dự phòng	Mã nguồn mở; cần máy chủ/agent và công bảo trì	Cao (Jenkinsfile, plugin)	Chạy cùng lệnh npm trên agent	Không tự triage trong phạm vi này	Tài liệu/plugin/cộng đồng rộng

E. Đề xuất công cụ

Chọn GitHub Actions + ChatGPT; Jenkins là phương án dự phòng nếu không thể dùng GitHub Actions.

- GitHub Actions tự chạy test và tạo status check trên PR.
- ChatGPT và người xem **cùng log/retry history** để so sánh công bằng.
- Jenkins có thể chạy cùng lệnh test nếu GitHub Actions không khả dụng. Để đáp ứng yêu cầu so sánh hai nền tảng của Topic 7, nhóm vẫn dự kiến chạy một lượt đối chiếu trên Jenkins khi GitHub Actions hoạt động (nếu được).

III. Nội dung dự định thuyết trình và demo

1. **Pipeline:** unit/API/smoke → cache npm, lint/build/Jest → coverage XML/artefact → badge, status check, branch protection.
2. **Demo CI:** PR chạy GitHub Actions; chạy cùng bộ test trên Jenkins để nêu khác biệt. Nếu GitHub Actions không khả dụng, dùng Jenkins làm pipeline chính và xin giảng viên duyệt cách đáp ứng mục tiêu so sánh.
3. **Demo AI:** sleep-based flake qua 10 lần chạy; người và ChatGPT xem cùng log, nêu giả thuyết + 3 bước kiểm tra, đối chiếu root cause.
4. **Hoạt động lớp:** “Pipeline Postmortem” 25 phút: xem YAML/log, lập cây giả thuyết 3 tầng, hỏi AI, công bố nguyên nhân.

IV. Nguồn tài liệu tham khảo

1. GitHub Actions — GitHub Docs
2. Jenkins Pipeline — Jenkins Docs
3. Use ChatGPT — OpenAI Docs
4. Prompting — OpenAI Docs
5. Pricing — OpenAI Docs

V. AI Disclosure

- **Công cụ và mục đích sử dụng: ChatGPT (Codex)** được dùng để gợi ý bố cục, soạn và chỉnh sửa phần khảo sát GitHub Actions, ChatGPT, Jenkins; rà soát sự nhất quán giữa bảng so sánh, lựa chọn công cụ và kế hoạch demo. AI cũng hỗ trợ viết ví dụ workflow, Jenkinsfile và prompt phân tích log.
- **Giai đoạn sử dụng AI:** [x] brainstorm
  [x] outline
  [x] viết nháp
  [x] phản hồi
  [x] sửa chữa
  [ ] code
  [ ] phân tích dữ liệu
  [ ] thiết kế đồ họa
  [ ] khác (ghi rõ)
- **Nhiệm vụ chính cho AI:** Đưa ra các liên kết đến tài liệu của các công cụ, tóm tắt nội dung và cung cấp minh chứng để kiểm chứng, chỉnh sửa.
- **Đóng góp của AI:**
  - Phần giới thiệu công cụ ChatGPT: AI đóng góp bộ khung và gợi ý prompt yêu cầu phân tích log. Phần còn lại nhóm tự sửa theo bản nháp.
  - Phần giới thiệu công cụ GitHub Actions và Jenkins: AI đưa ra bộ khung tìm hiểu. Nhóm dựa vào bộ khung đó kết hợp với tài liệu đã học trong môn Nhập môn DevOps để viết lại.
  - Phần nội dung Đề xuất và Nội dung dự định: AI có đóng góp trong quá trình phản hồi giữa các thành viên trong nhóm.
  - AI hỗ trợ sửa chính tả trong quá trình soạn thảo bị sai.

- **Cách rà soát/chỉnh sửa/xác minh đầu ra của AI:** Đối chiếu lại thông tin với tài liệu chính thức của các công cụ được liệt kê trong mục tài liệu tham khảo.